

Reflection Questions & Answers

Q1. Which optimization produced the greatest improvement?

Ans: Implementing thread pool management (ThreadPoolExecutor) in `server.py` produced the greatest improvement by preventing thread-creation overhead and allowing smooth handling of multiple concurrent client connections.

Q2. Why is scalability important?

Ans: Scalability ensures the server remains responsive, stable and efficient as traffic increases, preventing server crashes and high latency when many clients connect simultaneously.

Q3. What ~~real~~ reliability issues did you discover?

Ans: Unhandled socket timeouts caused client applications to freeze, sudden client disconnects left orphan threads on the server, and server crashes left clients in an unhandled hanging state without reconnection logic.

Q4. Which optimization was the most difficult?

Ans: Implementing graceful shutdown and thread-safe session cleanup, ensuring that active client sockets & threads were properly cleaned up upon server termination without causing race conditions.

Q5. What additional improvements are required to support 100 users?

Ans:- Transitioning from thread-based concurrency to an asynchronous I/O framework (like Python's `asyncio`), using a centralized database (such as PostgreSQL/Redis) instead of a CSV file for authentication, and implementing load balancing across multiple server instances.